

به نام خدا



درس سیستم عامل
نیم سال اول ۰۵-۰۴
استاد: دکتر اسدی

دانشکده مهندسی کامپیوتر

تمرین سری سوم

- پرسش‌های خود را در سامانه CW و تالار مربوط به تمرین مطرح نمایید.
- پاسخ سوالات را تایپ نمایید.
- اسکرین‌شات‌ها، عکس‌ها، فایل‌های مربوط به سوال عملی، گزارش تمرینات عملی و PDF قسمت تئوری را در پوشه با نام به فرمت HWNUM_StudentID1_StudentID2 ذخیره نمایید. سپس آن را zip نمایید و در صفحه درس بارگذاری نمایید.
- به عنوان مثال یک فایل بارگذاری شده قابل قبول باید دارای فرمت HW1_400123456_403123456.zip باشد.
- هر دانشجو می‌تواند حداکثر سه تمرین را با دو روز تأخیر بدون کاهش نمره ارسال نماید.
- تمرینات عملی و تئوری به صورت گروه‌های دو نفر تحویل داده شود.
- هر دو عضو گروه موظف هستند تمرینات خود را بارگذاری کنند.
- عواقب عدم تطابق بین پاسخ دو عضو گروه برعهده خودشان است.
- تحویل تمرین به صورت انگلیسی مجاز نیست. در صورت تحویل تمرین به صورت انگلیسی (حتی بخشی از تمرین) نمره تمرین موردنظر صفر در نظر گرفته می‌شود.
- در صورت مشاهده تقلب برای بار اول نمره هر دو طرف صفر می‌شود. در صورت تکرار نمره کل تمرینات صفر خواهد شد.
- استفاده از ابزارهایی مانند ChatGPT به منظور ابزار کمک آموزشی مجاز است به شرط آن که به خروجی آن اکتفا نشود.
- توجه شود که پروژه نهایی درس در گروه‌های چهار نفر تحویل گرفته می‌شود.
- سوالات با عنوان اختیاری نمره‌ای ندارند اما جواب دادن به آن‌ها کمک به‌سزایی در یادگیری درس می‌کند.
- هیچ تمرینی خارج از CW تصحیح نخواهد شد. لذا توصیه می‌شود حداقل یک الی دو ساعت قبل از زمان پایان تمرین برای بارگذاری تمرین اقدام شود.

تمارین تئوری

۱. به سؤالات زیر پاسخ دهید:

آ) چه ویژگی‌هایی از پدر به فرزند در اثر fork به ارث می‌رسد؟
 ب) یک پردازش P را در نظر بگیرید که پردازش فرزند C را در اثر fork ایجاد کرده است. حال P خاتمه می‌یابد در حالی که C هنوز در حال اجرا است. به سؤالات زیر پاسخ دهید:

• آیا C فوراً zombie می‌شود؟

• آیا P فوراً zombie می‌شود و تا زمانی که والدش آن را جمع‌آوری کند در حالت zombie باقی می‌ماند؟

ج) یک shell ساده لینوکس را در نظر بگیرید که فرمان sleep 100 را اجرا می‌کند. کدامیک از گزینه‌های زیر ترتیب فراخوانی‌های سیستمی را از زمانی که کاربر این فرمان را وارد می‌کند تا زمانی که shell دوباره آماده دریافت ورودی بعدی می‌شود، به درستی نشان می‌دهد؟ توضیح دهید.

• wait-exec-fork

• exec-wait-fork

• fork-exec-wait

• wait-fork-exec

۲. به سؤالات زیر پاسخ دهید:

آ) تمام دنباله‌های خروجی ممکن کد زیر را بنویسید.

```
1 int main()
2 {
3     int x = 3;
4     if (fork() != 0)
5         printf("x=%d\n", ++x);
6     printf("x=%d\n", --x);
7     return 0;
8 }
```

ب) در کد زیر، هنگام فراخوانی funfork() چند خط خروجی مشاهده می‌شود؟ عدد را بر حسب n بنویسید (با فرض اینکه n بزرگتر مساوی ۱ است).

```
1 void funfork(int n) {
2     int i;
3     for (i = 0; i < n; i++)
4         fork();
5     printf("hi\n");
6     exit(0);
7 }
```

ج) دنباله‌های خروجی ممکن کد زیر کدامند؟ تمام دنباله‌های خروجی ممکن را بنویسید.

```
1 int main() {
2     if (fork() == 0) {
3         printf("a"); fflush(stdout);
4     } else {
5         printf("b"); fflush(stdout);
6         waitpid(-1, NULL, 0);
7     }
8     printf("c"); fflush(stdout);
9 }
```

```

9   exit(0);
10  }

```

د) کدام یک از گزینه‌های زیر می‌تواند خروجی کد زیر باشد؟ با توضیح جواب خود را توجیه نمایید. توجه کنید که `atexit()` اشاره‌گری به یک تابع می‌گیرد و آن را به فهرستی از توابع (که در ابتدا خالی است) اضافه می‌کند. این توابع هنگام فراخوانی `exit()` به ترتیب معکوس ثبت، اجرا می‌شوند.

- (آ) ۱۱۲۰۰۲
- (ب) ۲۱۱۰۲۰
- (ج) ۱۰۲۱۲۰
- (د) ۱۲۲۰۰۱
- (ه) ۱۰۰۲۱۲

```

1  void end(void)
2  { printf("2"); fflush(stdout); }
3
4  int main()
5  {
6      if (fork() == 0) atexit(end);
7      if (fork() == 0) {
8          printf("0"); fflush(stdout);
9      } else {
10         printf("1"); fflush(stdout);
11     }
12     exit(0);
13 }

```

۳. در یک `shell` یونیکس، دستور `A <> B` به این معناست که خروجی استاندارد برنامه‌ی A باید به ورودی استاندارد برنامه‌ی B وصل شود و همزمان ورودی استاندارد برنامه‌ی A نیز به خروجی استاندارد برنامه‌ی B متصل شود، به عبارتی یک خط لوله‌ی دوطرفه بین این دو برنامه ایجاد می‌کند. کد C ارائه شده بخشی از پیاده‌سازی این `shell` است اما دارای چندین خطا است که باعث می‌شوند این رفتار به درستی انجام نشود. در این سؤال، لازم است شماره‌ی خطوطی که اشتباه دارند مشخص شود، به‌طور مختصر مشکل هر خط توضیح داده شود و راه‌حلی برای برطرف کردن آن ارائه شود تا کد بتواند اتصال دوطرفه‌ی ورودی و خروجی بین دو برنامه را به درستی پیاده‌سازی کند.

```

1  #define WRITE 2
2  #define READ 0
3
4  int pipe1 [2] , pipe2 [1];
5
6  pipe ( pipe1 );
7  pipe ( pipe2 );
8
9  if( fork () == 0 ) {
10     dup2 ( pipe1 [ WRITE ], STDOUT );
11     dup2 ( STDIN , pipe2 [ READ ] );
12     close ( pipe1 [ READ ] );
13     close ( pipe1 [ WRITE ] );
14     exec ( A );
15 }
16
17 if( fork () != 0 ) {
18     dup2 ( pipe1 [ READ ], STDIN );
19     dup2 ( pipe2 [ WRITE ], STDOUT );

```

```

20     close ( pipe1 [ WRITE ] ) ;
21     close ( pipe2 [ READ ] ) ;
22     exec (B);
23 }
24
25 close ( pipe1 [ READ ] ) ;
26 close ( pipe1 [ WRITE ] ) ;
27 close ( pipe2 [ READ ] ) ;
28 close ( pipe2 [ WRITE ] ) ;

```

۴. قطعه کد زیر را در نظر بگیرید:

```

1  #include <stdio.h>
2  #include <unistd.h>
3
4  int main() {
5      pid_t p1, p2;
6
7      printf("X\n");
8
9      p1 = fork();
10
11     if (p1 > 0) {
12         printf("Y\n");
13         p2 = fork();
14         if (p2 == 0) {
15             printf("Z\n");
16             fork();
17             fork();
18             fork();
19         }
20     } else {
21         printf("W\n");
22         fork();
23         fork();
24         fork();
25     }
26
27     printf("K\n");
28
29     return 0;
30 }

```

آ) درخت پردازها را برای قطعه کد فوق رسم کنید.

ب) عبارت X چند مرتبه و عبارت K چند مرتبه چاپ می‌شود؟

ج) با توجه به همزمانی اجرا و عدم وجود دستور wait دو دنباله خروجی ممکن (برای مثال XYKWZK...) را که توسط این قطعه کد تولید می‌شود، بنویسید.

۵. به سؤالات زیر پاسخ دهید:

آ) مشخص کنید که هر یک از پردازهای زیر در کدام یک از حالات در حال اجرا، آماده و در انتظار قرار دارد؟

- پردازهای که در چرخش انتظار برای صفر شدن یک متغیر است.
- پردازهای که تازه عملیات ورودی/خروجی را تمام کرده و منتظر است دوباره در پردازنده زمان‌بندی شود.

● پردازهای که در حال پیمایش بافر حافظه نهان در هسته برای پاسخ به read است.

ب) سناریوهای زیر را در نظر گرفته و به سؤالات پاسخ دهید.

● اگر یک پردازش فرزند سیگنال SIGKILL را به پردازش والد خود ارسال کند، چه اتفاقی برای پردازش والد و پردازش فرزند می‌افتد؟ آیا پردازش والد می‌تواند از وقوع این اتفاق جلوگیری کند؟

● یک پردازش، یک پردازش فرزند ایجاد می‌کند (با استفاده از fork()) و سپس پردازش والد برای آن منتظر می‌ماند (wait()). در ادامه، پردازش فرزند به صورت عادی خاتمه می‌یابد و پردازش والد از حالت انتظار خارج می‌شود. در نهایت، پردازش والد دوباره یک فراخوانی wait() روی همان پردازش فرزند انجام می‌دهد. چه اتفاقی برای این فراخوانی wait() دوم و برای خود پردازش والد می‌افتد؟

ج) درستی یا نادرستی موارد زیر را مشخص کنید و توضیح دهید.

● اگر یک پردازش را fork کنید، تغییراتی که در توصیف‌گرهای فایل ایجاد می‌شوند، در هر دو پردازش والد و فرزند اثر می‌گذارند.

● یک پردازش فرزند می‌تواند با والد خود از طریق یک ساختار داده در heap خود که قبل از فراخوانی سیستم fork() توسط والد تخصیص داده شده است، ارتباط برقرار کند.

به سؤالات زیر پاسخ دهید:

۶. آ) با توجه به کد زیر، مقادیر pid را در خط‌های A، B، C و D مشخص کنید. فرض کنید pidهای پدر و فرزند به ترتیب ۲۶۰۰ و ۲۶۰۳ هستند.

```

1 #include <sys/types.h>
2 #include <stdio.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 int main()
7 {
8     pid_t pid, pid1;
9
10    /* fork a child process */
11    pid = fork();
12
13    if (pid < 0) {
14        /* error occurred */
15        fprintf(stderr, "Fork Failed\n");
16        return 1;
17    }
18    else if (pid == 0) {
19        /* child process */
20        pid1 = getpid();
21        printf("child: pid = %d\n", pid);    /* A */
22        printf("child: pid1 = %d\n", pid1); /* B */
23    }
24    else {
25        /* parent process */
26        pid1 = getpid();
27        printf("parent: pid = %d\n", pid);    /* C */
28        printf("parent: pid1 = %d\n", pid1); /* D */
29        wait(NULL);
30    }
31
32    return 0;
33 }
```

ب) با توجه به کد زیر، خروجی خط‌های X و Y چه خواهد بود؟

```

1 #include <sys/types.h>
2 #include <stdio.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 #define SIZE 5
7
8 int nums[SIZE] = {0, 1, 2, 3, 4};
9
10 int main()
11 {
12     int i;
13     pid_t pid;
14
15     pid = fork();
16
17     if (pid == 0) {
18         /* Child process */
19         for (i = 0; i < SIZE; i++) {
20             nums[i] *= -i;
21             printf("CHILD: %d ", nums[i]); /* LINE X */
22         }
23         printf("\n");
24     }
25     else if (pid > 0) {
26         /* Parent process */
27         wait(NULL);
28         for (i = 0; i < SIZE; i++) {
29             printf("PARENT: %d ", nums[i]); /* LINE Y */
30         }
31         printf("\n");
32     }
33
34     return 0;
35 }

```

ج) پس از اجرای قطعه کد زیر، عبارت Hello World و عبارت Goodbye World چند بار چاپ می‌شوند؟ در مورد علت پاسخ خود توضیح دهید (برای این کار می‌توانید از درخت پردازش‌ها کمک بگیرید).

```

1 fork() ? (fork() && fork()) | (fork() & fork()) : printf("\n");
2 if (fork() && (!fork())) {
3     if (fork() || fork()) {
4         printf("Goodbye World!\n");
5     }
6 }
7 printf("Hello World!\n");

```

تمارین عملی

۱. XV6 OS

در این تمرین می‌خواهیم دستور top تمرین قبل را بهبود ببخشیم و روابط پدر-فرزندی را نیز در آن نمایش دهیم. این فراخوانی سیستمی باید به صورت زیر باشد:

```
1 struct process_data {
2     int pid;
3     int parent_id;
4     int head_size;
5     enum procstate state;
6     char name[16];
7 };
8 int next_process(int before_pid, struct process_data* proc)
```

کاری که این فراخوانی سیستمی باید انجام دهد این است که اطلاعات اولین پردازش‌های را برگرداند که process ID آن از before_pid بیشتر باشد. در صورتی که پردازش‌های وجود داشت که process ID آن بیشتر از before_pid باشد، باید این تابع مقدار ۱ را برگرداند و اطلاعات پردازش را در proc قرار دهد و در غیر این صورت مقدار ۰ را برگرداند و proc را دست نزنند.

۱. در ابتدا باید یک گزینه به دستور top که در تمرین قبل زدید، اضافه کنید تا با اجرای آن، خروجی دستور top به صورت درختی باشد و روابط پدر-فرزندی پردازش‌ها را نمایش دهد.

```
1 top -t
2 top --tree
```

۲. در هنگام اجرای این دستور علاوه بر خواسته‌های تمرین دوم، باید وضعیت آن پردازش (Running، Sleeping، Runnable، Zombie) را نیز نشان دهید.

۳. در ادامه یک user program بنویسید که درخت پردازش‌هایی که در سیستم در حال اجرا هستند را رسم کند. برنامه‌ی خود را با استفاده از تابع fork و exec در یک برنامه‌ی دیگر تست کنید.

دقت کنید که در نهایت نیز یک گزارش مختصر همراه با اسکرین‌شات از اجرای برنامه‌ی تست خود تهیه کنید و همراه با کدهایی که تغییر داده‌اید و یا نوشته‌اید در سامانه cw بارگذاری کنید.

راهنمایی: می‌توانید از `before_pid = 0` شروع کنید و آن قدر تابع `next_process` را با `pid` برگردانده شده صدا کنید که خروجی آن ۰ شود. دقت کنید که برای رسم درخت پردازش‌ها، شما باید روابط پدر به فرزند را داشته باشید اما خروجی این فراخوانی سیستمی، روابط فرزند به پدر را نگه می‌دارد. برای حل این مشکل می‌توانید لیست تمامی پردازش‌ها را در `userspace` نگه دارید. دقت کنید که این سیستم عامل نمی‌تواند مانند لینوکس به صورت همزمان دو خط فرمان را اجرا کند پس برای تست کارکرد برنامه‌ی خود باید به کمک دستور `fork` چندین پردازش ایجاد کنید و در نهایت قبل از بستن تمامی آن‌ها، برنامه‌ی خود را اجرا کنید. می‌توانید به کمک تابع `malloc` یا `sbrk` حافظه `heap` از سیستم عامل درخواست کنید. همچنین یک فیلم آموزشی نیز برای راحتی کار شما در cw درس قرار داده شده است.

اختیاری: سیستم عامل لینوکس از چه روشی برای گرفتن لیست پردازش‌ها استفاده می‌کند؟ سیستم عامل ویندوز چگونه این کار را انجام می‌دهد؟

اختیاری: یکی از روش‌هایی پاس دادن اطلاعات پردازش‌ها به برنامه کاربر به این صورت است که یک آرایه از `struct process_data` برگردانیم، چالش‌های آن را بیان کنید.

۲. Docker

در سری تمرینات داکر قصد داریم که به تدریج اجزایی از داکر را پیاده‌سازی کنیم و در راستای آن مفاهیم مربوط به داکر را از دیدگاه سیستم عامل بررسی کنیم. فایل تمرینات در این مخزن قرار خواهد گرفت.

اگر از تمرین ۱ به یاد داشته باشید، در پایان آن مشاهده‌ای در مورد PID پردازه‌ی اصلی کانتینر داشتیم. در داخل کانتینر PID پردازه برابر با ۱ نشان داده می‌شد، در حالی که در خارج کانتینر PID همان پردازه، عدد متفاوتی بود. این مسئله نتیجه وجود مفهومی در سطح سیستم عامل تحت عنوان فضای نام^۱ است. برای مطالعه بیشتر این مفهوم در سیستم عامل manpage این را مطالعه کنید.

هر پردازه‌ها از چندین تشکیل شده است. در این تمرین ما تنها تمرکز روی فضای نامی PID داریم. در این فضای نام PID پردازه‌ها از سایر پردازه‌های سیستم عامل جدا خواهد شد. به این معنا که مانند سیستم عامل، اولین پردازه در یک فضای نامی جدید، مشابه پردازه init عمل می‌کند و مقدار PID آن برابر ۱ خواهد بود. در این تمرین قصد داریم که یک پردازه را در فضای نامی جدید اجرا کنیم.

گام‌های تمرین

آ. محتوای تکلف ۱ را از مخزن تمرین دریافت کنید و پروژه را با دستور make کامپایل کنید.

ب. دستور زیر را اجرا کنید.

```
1 ./zocker run --name hello 'readlink /proc/self/ns/pid'
```

سوال: این دستور چه چیزی را دقیقاً نشان می‌دهد؟ در رابطه با محتوای موجود در مسیر /proc/<pid>/ns برای پردازه‌ها به صورت مختصر توضیح دهید.

ج. **سوال:** دو رویکرد کلی clone(2) و fork(2) + unshare(2) برای اجرای یک پردازه فضای نامی جدید وجود دارد. به صورت کوتاه این دو روش را بررسی کنید و توضیح دهید که چرا روش دوم برای پیاده‌سازی انتخاب آسان‌تری است؟

حال با استفاده از روش fork(2) + unsahre(2) محتوای داخل تابع run_container را به نحوی تغییر دهید که PID پردازه جدید در خروجی ۱ نمایش داده شود و همچنین خروجی خود دستور اجرا شده نیز متفاوت از حالت عادی شود و شناسه‌ی جدیدی برای فضای نامی PID نشان بدهد. (به احتمال بالا پس از انجام تغییرات اجرای برنامه به صورت مشابه دچار اشکال شود و نیازمند sudo برای اجرای درست باشید).

د. **سوال:** چرا پس از پیاده‌سازی بخش قبل نیاز به sudo هنگام اجرا داریم؟ (بخش CLONE_NEWPID این مستند را مطالعه کنید).

حال Makefile را به نحوی تغییر دهید که اجرای باینری zocker بدون sudo نیز ممکن باشد. (راهنمایی: در رابطه با CAP_SYS_ADMIN و دستور setcap مطالعه کنید).

ه. (اختیاری) در رابطه با setns(2) تحقیق کنید و با استفاده از آن دستور exec را با اضافه کردن پرچم‌های مورد نیاز تکمیل کنید.

مواردی که باید تحویل داده شود:

- پاسخ به سه سوال.
- تصویر خروجی اجرای بخش ج.
- کد تغییر یافته.

¹Namespace

۳. توسعه فراخوانی سیستمی

هدف تمرین

این تمرین با هدف آشنایی با مفاهیم زیر طراحی شده است:

- درک معماری فراخوانی‌های سیستمی در لینوکس
- پیاده‌سازی و آزمایش فراخوانی‌های سیستمی سفارشی
- آشنایی با ساختار **PCB** در لینوکس
- تغییر و توسعه هسته لینوکس

مراحل تمرین

۱. فراخوانی سیستمی اولیه

۱. ایجاد یک فراخوانی سیستمی جدید که پیام `hello world` را در سطح هسته چاپ می‌کند
۲. نوشتن یک برنامه سطح کاربر برای فراخوانی این فراخوانی سیستمی
۳. مشاهده خروجی با استفاده از دستور `dmesg`

۲. توسعه فراخوانی سیستمی

۱. تغییر فراخوانی سیستمی به طوری که پیام را به خروجی استاندارد چاپ کند
۲. آزمایش و تأیید عملکرد صحیح

۳. تحقیق درباره PCB

۱. مطالعه و تحقیق در مورد ساختار **Process Control Block** در لینوکس
۲. شناسایی محل پیاده‌سازی آن در کد منبع هسته
۳. مستندسازی یافته‌ها

۴. توسعه ساختار PCB

۱. افزودن یک فیلد جدید به ساختار **PCB**
۲. پیاده‌سازی دو فراخوانی سیستمی:
 - فراخوانی برای خواندن مقدار فیلد جدید
 - فراخوانی برای نوشتن مقدار در فیلد جدید

۵. توسعه ساختار PCB

۱. افزودن یک فیلد جدید به ساختار **PCB**
۲. پیاده‌سازی دو فراخوانی سیستمی:
 - فراخوانی برای خواندن مقدار فیلد جدید
 - فراخوانی برای نوشتن مقدار در فیلد جدید

۶. آزمایش و ارزیابی

۱. آزمایش کلیه موارد پیاده‌سازی شده
۲. اطمینان از عملکرد صحیح تمامی بخش‌ها
۳. مستندسازی نتایج

خروجی مورد انتظار

- درک عمیق از معماری فراخوانی‌های سیستمی
- توانایی در تغییر و توسعه هسته لینوکس
- مستندات کامل از مراحل پیاده‌سازی
- کدهای منبع پیاده‌سازی شده

۲. اندازه‌گیری زمان فراخوانی‌های سیستمی

هدف تمرین

این تمرین با هدف اندازه‌گیری و تحلیل زمان‌بندی فراخوانی‌های سیستمی طراحی شده است:

- اندازه‌گیری زمان رسیدن درخواست به کد اصلی فراخوانی سیستمی
- اندازه‌گیری زمان بازگشت از فراخوانی سیستمی به برنامه کاربر
- مقایسه زمان‌های فراخوانی‌های سیستمی مختلف
- تحلیل کارایی و سربار فراخوانی‌های سیستمی

مراحل تمرین

۱. پیاده‌سازی فراخوانی سیستمی زمان‌سنج

۱. ایجاد یک فراخوانی سیستمی جدید که زمان کنونی سیستم را بازمی‌گرداند
۲. ثبت فراخوانی سیستمی در جدول فراخوانی‌های سیستمی هسته
۳. استفاده از توابع زمان‌سنجی با دقت بالا در هسته

۲. اندازه‌گیری زمان فراخوانی

۱. اندازه‌گیری زمان قبل از ورود به فضای هسته
۲. اندازه‌گیری زمان بلافاصله پس از ورود به تابع فراخوانی سیستمی
۳. اندازه‌گیری زمان قبل از خروج از فضای هسته
۴. اندازه‌گیری زمان پس از بازگشت به فضای کاربر
۳. آزمایش با فراخوانی‌های سیستمی مختلف

۱. انجام آزمایش با فراخوانی سیستمی read

۲. انجام آزمایش با فراخوانی سیستمی write

۳. انجام آزمایش با فراخوانی سیستمی getpid

۴. انجام آزمایش با فراخوانی سیستمی سفارشی ایجاد شده

۴. تحلیل و گزارش نتایج

۱. محاسبه زمان صرف شده در مسیر رفت (User to Kernel)

۲. محاسبه زمان صرف شده در مسیر برگشت (Kernel to User)

۳. محاسبه زمان کل فراخوانی سیستمی

۴. مقایسه و تحلیل نتایج به دست آمده